

A Reactive Metadata Indexing Daemon for Linux

Michael Connor Buchan

3110037

Supervised by: Dr Patrick Maier

April 2026

**Dissertation submitted in partial fulfilment for the degree of
Bachelor of Science
with Honours Computing Science**

Computing Science and Mathematics
University of Stirling

Abstract

Problem: File search indexing is notoriously hard to keep accurate and up to date with respect to the canonical file system metadata that it indexes. Recent Linux kernels present new APIs that may help to create an indexing daemon that can more quickly react to file system events and index richer file metadata.

Objectives: This project introduces indexd, a reactive file metadata indexing daemon for Linux that leverages the fanotify event notification API, entire file system monitoring and rich statx and extended attribute metadata, to build and maintain an index, which can support future query engines to efficiently find files based on common and arbitrary metadata values.

Methodology: Specific attention was paid to reducing the daemon's resource footprint, so that it may be used on battery powered consumer devices, such as smartphones and laptops. The daemon makes use of the Lightning Memory-mapped Database Manager (LMDB) to achieve this by allowing the shared kernel page cache to manage database memory.

Achievements: Overall the daemon fulfills most of its objectives and can successfully react to metadata change events and update its index accordingly. It also constructs a correct index via a manual hierarchy walk on startup.

Attestation

I understand the nature of plagiarism, and I am aware of the University's academic integrity policy.

I certify that this dissertation reports original work by me during my honours project except for the following:

- Files with leading comments disclosing the use of generative AI, with accompanying transcripts

Signature Michael Connor Buchan **Date** 18/04/2026

Acknowledgements

I would like to thank .. my supervisor, Patrick Maier, as well as my beloved mother, Joanne Buchan, for helping me edit this document.

Contents

Abstract	1
Attestation	2
Acknowledgements	3
1 Introduction	7
1.1 Background and Context	7
1.2 Problem Analysis	8
1.3 Scope and Objectives	10
2 State of the Art	11
2.1 Desktop Search Systems	11
2.1.1 Locate	11
2.1.2 TinySPARQL and Gnome LocalSearch	11
2.2 File System Monitoring Techniques	12
2.2.1 Dnotify	12
2.2.2 Inotify	13
2.2.3 Fanotify	13
2.2.4 eBPF	14
3 Development	15
3.1 System Requirements	15
3.1.1 Functional Requirements	15

3.1.2	Non-Functional Requirements	17
3.2	High-Level Architecture	17
3.2.1	Concurrency Model	19
3.3	File System Scanning and Monitoring	20
3.4	Storage Engine	21
3.5	Data Model	22
3.5.1	File Identity	22
3.5.2	Reverse Index Design	23
3.5.3	Forward Index Design	23
3.5.4	Index Maintenance	23
4	Evaluation	25
4.1	Evaluation Methodology	25
4.2	Experimental Setup	26
4.3	Correctness Results	29
4.4	Performance Results	29
4.4.1	CPU Usage	30
4.4.2	Context Switches	31
4.4.3	Memory Page Faults	32
4.5	Critical Objective Evaluation	32
4.5.1	Change detection	32
4.5.2	Storage format	33
4.5.3	Manual scan	33
4.5.4	Index reactivity	33
4.5.5	Correctness	34
4.5.6	Performance	34
5	Conclusion	35
5.1	Future Work	35

List of Figures

- 3.1 High-level architecture of the indexer daemon, showing the
main and writer threads and their responsibilities. 18

Chapter 1

Introduction

The aim of this project is to provide a background service for Linux systems that builds an index of file metadata and attributes, and proactively maintains its index based on methods of file system event notification provided by the Linux kernel. This index may then be used by other tools to efficiently find files based on their characteristics such as location, creation and modification timestamps, or user / application supplied extended attributes such as tags or categories.

1.1 Background and Context

In computing, files are numerous and everywhere. The backbone of permanent data storage in most general-purpose systems, they can store any kind of data. Over time, in any given system, the number of files grows substantially, and it becomes important to be able to categorise, sub-divide, monitor, and audit collections of these files.

Files are accompanied by metadata: additional information about the files themselves, such as their creation, modification, and access times; who has permission to view and edit them; and even user or application specific information, such as where a file originated from (eg. a website URL or email message), or a list of user defined tags to make it easier to find related files. This rich metadata offers important insight into how files are used, what they are for, and what their value is to the user or automated system working with them:

- On the desktop, a user may use a search tool to find files which were modified in a given time interval; which were downloaded from a particular website; or another arbitrary list of criteria, to find files they

need more easily.

- Cloud storage systems may require similar solutions.
- Backup services require a list of files that have changed since the last backup was completed.
- Security auditing tools need to know when file access permissions change, and why, so this information can be recorded in an audit log.
- Anti-virus scanners and other security protections should check whether file access permissions were relaxed by rogue or malicious programs during their periodic scans, so users can be alerted and / or the permissions can automatically be fixed.

This information is only useful in many of these contexts if it can be queried in an efficient manner. Many current implementations of the above tools implement their own methods of manually crawling the file system, either to find files that match certain criteria, or to detect when file attributes have changed. This can be very computationally inefficient, particularly when the file system is large (many thousands of files). Lacking a centralised, structured index of this metadata may also lead to duplicated effort, as multiple programs must update their own, siloed metadata databases.

The “freshness” of this information is likewise vital to tools that rely on it. Traditional file system crawlers often miss intermediate changes to attributes and may take some time to surface metadata changes. If, for example, the user has created several files since the last reindexing operation, and now wants to find them, the system does not know of the new files yet, and the user is left wondering why their files are not appearing in the search results. Lacking timely notifications about file attribute changes can be especially damaging for security tools, as they may not detect a change in file access permissions until an attacker has already accessed sensitive or confidential information.

While POSIX employs a broader definition of a “file” as “An object that can be written to, or read from, or both,” [6, §3.139 “File”] this report defines the term more narrowly as an object stored on a file system whose metadata may be queried. This includes, but is not limited to: regular files, directories, symbolic links, FIFOs (named pipes), sockets and device nodes.

1.2 Problem Analysis

In a standard Posix file system, finding files based on their location within the directory hierarchy is a fast operation, supported by years of optimisation

in the kernel’s path resolution code. [20, 1] However, finding files based on more complex criteria, such as their content; fixed attributes such as creation, modification, and access times; or free-form attributes such as the extended attributes (xattrs) feature common to several Unix variants, is usually much more resource intensive. Most file systems do not provide efficient on-disk data structures for enumerating these attributes, nor the higher-level interface for constructing and executing queries against them to find files.

Metadata indexing services were created to fill this gap and are often used in contexts where such queries would be useful. They create and maintain their own independent index, mapping values of file attributes to the files they belong to. Since this index is not part of the underlying file system, it must be manually kept up to date, to reflect changes to file metadata, or the files themselves (such as when a file is created, deleted, or renamed).

An indexing service should be designed to have low overall resource usage, in order to minimise the impact of the service on foreground application performance. Many current indexers are resource intensive as they periodically rewalk entire file systems to detect changed file metadata. This can negatively impact foreground application performance in a few ways:

1. **Increased I/O latency and queue depths**, since they use a fairly large portion of a disk’s I/O bandwidth, reducing the bandwidth available to other processes due to channel congestion. This is particularly noticeable on spinning hard drives, as head seek times between index and foreground data further compound I/O latency.
2. **Kernel page cache pollution**, evicting hot data from files needed by foreground applications and instead filling potentially the entire cache with cold metadata from unrelated files.

For these reasons, it is important that the indexing service take a reactive approach to metadata updates, using operating system specific interfaces to watch for changes in files or their metadata: Linux provides several kernel interfaces for file system event notification. However, most existing indexing services, use an older, more limited subsystem called inotify [11]. This interface works well for monitoring a small number of files and directories. However, it does not scale well, becoming less efficient and less dependable on much larger datasets (such as a desktop user’s entire home directory). Despite being created for entirely different purposes, newer Linux technologies such as fanotify [8] offer a significant opportunity to enhance the efficiency and flexibility of indexing services, so this project explores this possibility.

Another important consideration for an indexing service is the structure of its index, as this correlates directly with the efficiency of queries performed

against said index. The index's structure should be specifically designed to be maximally efficient for the kinds of queries that the user (or client applications) will perform; this project researches and designs such a structure.

1.3 Scope and Objectives

The project focuses on two key aspects of the design of a core file metadata indexing service:

- **File system event notification:** how the service is notified when file metadata changes,
- **Indexed metadata storage:** how the service stores this updated metadata for hypothetical retrieval by other applications down the line.

This project's objectives are restated to focus on outcomes rather than the development process, but the core goals remain the same. They are as follows:

1. **Change detection:** Provide a mechanism for detecting file system events that describe or imply changes that affect the metadata index, such as file creation, deletion, renaming and metadata modification, as well as extended attribute changes.
2. **Storage format:** Design and implement an index database and schema, suitable for efficient queries to find files with matching metadata, and for reactive maintenance when metadata changes.
3. **Manual scan:** Implement an initial indexing process that fills the index by crawling the directory tree whose contents are indexed and monitored by the indexer (the **monitored hierarchy**), at startup. This is necessary as a pre-filled index is needed to build and evaluate change detection.
4. **Index reactivity:** Maintain the index as the underlying file system state changes, reacting to updated metadata in a timely manner.
5. **Correctness:** Evaluate index correctness after mutation by comparing it against canonical file system information, paying attention to correct updates for different event types regardless of whether files are already indexed.
6. **Performance:** Evaluate system efficiency by measuring the solution's CPU, memory, and I/O footprint, compared to the initial crawling process.

Chapter 2

State of the Art

2.1 Desktop Search Systems

Several existing indexing daemons are available on Linux.

2.1.1 Locate

The grandfather of all indexed Unix file search utilities, `locate(1)` “started life as the BSD fast find program, contributed to BSD by James A. Woods.” [4] Unlike the pre-existing `find` command, which traverses the directory hierarchy on demand, `locate` “consults a pre-processed database, regenerated daily.” [18] This approach reduced the time taken to find a file by name by half.

While `locate` provides a performance boost for path-matching based queries, it is less flexible than other search utilities, as it supports only partial matches on absolute file paths and does not index any other file metadata. Moreover, because `locate` foregoes detecting changes to files in real time in favour of rescanning the file system on a schedule, it reacts slowly to file creation, rename and delete operations, making it unsuitable for applications where changes need to be reflected in the index immediately.

2.1.2 TinySPARQL and Gnome LocalSearch

TinySPARQL (formerly Gnome Tracker) is the Gnome desktop’s take on “an efficient search engine and distributed database for desktop, embedded and mobile.” [5] It comprises a highly modular set of libraries and daemons. The core index stores data in RDF data structures and queries are written in the high level SPARQL query language.

TinySPARQL itself offers only the content indices and query engine; content discovery and ingestion are delegated to minors: auxiliary daemons that find and index content within the distributed RDF database. Local file system indexing is performed by Gnome's LocalSearch daemon (formerly tracker-minor-fs), which performs occasional rescans and uses either inotify [11] or fanotify [8] depending on kernel support, to track file system changes. File metadata is extracted not from extended attributes, but directly from file contents, using pluggable extractors per file-type.

This architecture delivers rich, expressive content storage and query capabilities at the cost of complexity. While it is well suited to desktop search, this project targets an indexing approach with lower operational complexity, focused tightly on extending existing file system capabilities, and with a smaller CPU and memory footprint allowing it to be adaptable to other environments, such as resource constrained embedded systems and servers. LocalSearch also creates full text search indices of documents, which provides more relevant desktop search results, but is much more resource intensive than attribute indexing and may not be necessary in some contexts.

2.2 File System Monitoring Techniques

2.2.1 Dnotify

The first file system event notification API added to the Linux kernel, Dnotify is superseded by inotify due to poor design decisions. Interest in events is registered with an `fcntl()` call on a directory file descriptor, and events are delivered via Unix signals. [17] The shortcomings of this design are numerous.

- Directories must be held open by applications monitoring them, which also holds open the file system mount on which they reside, making this interface impractical for use to monitor directories on removable or hot-pluggable media. Furthermore, since Dnotify does not deliver events for nested sub-directories, one file descriptor is required per monitored directory, and applications may quickly run into open FD limits enforced by the kernel.
- The use of signals to deliver events limits the amount of information that can be provided to applications. For example, when a file is created, Dnotify only informs the application that one or more files have been created; to find out which files were created, the application must manually rescan the directory.

- Dnotify also suffers from the limitations of its successor interface, inotify.

For these reasons, Dnotify is not suitable for a modern file metadata indexing application like this project.

2.2.2 Inotify

Inotify addresses many of the issues of its predecessor API by using one special file descriptor created using `inotify_create()`, and a separate namespace of watch descriptors, which are not tied to open file descriptions. Unlike the lack of information provided by Dnotify, inotify provides an ordered stream of detailed events about watched files, including paths to the files that have changed. [11]

This design solves the previously discussed deficiencies of Dnotify and makes inotify suitable as a general-purpose FS monitoring interface (indeed it is used by many file indexers if more modern interfaces are not available). However, there are still shortcomings which impede inotify’s scalability:

- Directories are still not recursively monitored, so the application must manually register interest in newly created directories to receive events from them. This operation is inherently prone to race conditions: “by the time you create a watch for the new subdirectory, new files (and subdirectories) may already exist inside the subdirectory,” [11]
- Because directories are watched individually, the amount of book-keeping required to monitor a large directory hierarchy is substantial, and registering watches for such a tree is slow.

These limitations make inotify suitable for project prototyping, but less than ideal for a production indexer.

2.2.3 Fanotify

Introduced in Linux 2.6.36 and extended to support more event types in Linux 5.1, fanotify [8] improves on inotify for the purposes of tracking file metadata changes by allowing an application to receive events for all files, directories, and other objects in a file system, provided that it is mounted. This solves the scalability problems of inotify, as watches do not need to be maintained per directory. As such, it is more reliable and is thus the preferred method of FS event notification for LocalSearch (see 2.1.2 above).

This interface still has a few limitations:

- Events are only reported if they were caused by userspace applications. If, for example, the in-kernel NFS server creates a new file, this will not be reported by fanotify.
- Like inotify, events may be lost if the event queue overflows, such as during heavy system load. [8]

Despite these limitations, it remains the best special purpose interface for file system notification in current kernels.

2.2.4 eBPF

The Enhanced Berkeley Packet Filter (eBPF) subsystem of the kernel offers a fundamentally different method of file system event notification. eBPF is a kernel virtual machine that allows userspace applications to run small programs in the privileged context of the kernel. [3] These eBPF programs can hook into many kernel subsystems using existing kernel infrastructure such as tracepoints, and can communicate with their host userspace application using eBPF maps such as ring-buffers.

This technology presents the opportunity for a search indexer application to load an eBPF program that tracks system calls pertaining to file system events and relays these events back to userspace. eBPF provides several ways to trace system calls. [14]

This approach may allow a metadata indexer to capture kernel function calls that perform file system modifications more reliably than traditional event notification mechanisms; for example, if hooks are added directly at the VFS layer, modifications performed on behalf of networked file systems could still be observed.

Chapter 3

Development

3.1 System Requirements

A set of functional and non-functional requirements were derived from the previously stated objectives, to determine the characteristics needed to realise the project's goals.

3.1.1 Functional Requirements

The solution requires the following capabilities to achieve its objectives:

- **Initial scan:**

The daemon must traverse the monitored hierarchy on startup, constructing the initial index from the metadata of each visited file. If started with an index that already contains data, it should update the metadata of scanned files to reflect the current file system state.

- **File creation detection:**

The daemon must detect the creation of files in the monitored hierarchy, collecting their metadata from the file system.

- **File deletion detection:**

The daemon must detect the removal of files from the monitored hierarchy.

- **File rename detection:**

The daemon should detect renamed files or files moved internally within the monitored hierarchy. Files moved into the hierarchy from

an external location should be treated as created; files moved out of the hierarchy should be treated as deleted.

- **File attribute change detection:**

The daemon must detect when a file's metadata changes. Monitored metadata should include file size, modification time (mtime), metadata change time (ctime) and a user-configurable set of extended attributes. Additional attributes may be supported.

- **Index reactivity:**

The daemon must respond to newly discovered files and changed file metadata by inserting current metadata into the index, updating/replacing existing records where stale metadata already exists. Deleted files should be handled by finding and removing index records associated with that file. Deletion operations should be idempotent: deleting a file with no indexed metadata should do nothing. Where files are renamed or moved, index records should be reassociated with the new path; if the index does not contain the file already, the daemon should treat this event as if the file was created and insert index records for its current metadata.

- **Index reconciliation:**

When updating index records, the daemon must ensure that the current state of the file system is accurately reflected for each changed file. This includes insertion of new xattr values, updating of xattrs whose values have changed and removal of stale xattr entries that are no longer present.

- **ACID index guarantees:**

The index must reside in a durable database format so that committed changes persist after abnormal process termination.

When the index is updated from current file system state, records pertaining to each file must be updated as atomically as one logical operation, in order to prevent the index from entering an inconsistent state. It must be impossible to observe a state in which only **some** of a file's metadata has been updated in the index.

Index data structures should also remain mutually consistent: records referencing a file by its identifier must all refer to the same file, to preserve referential integrity.

3.1.2 Non-Functional Requirements

The solution is constrained by the following non-functional requirements, which describe the desired qualities required to satisfy the project objectives:

- **Index correctness:**

The index should remain as close as possible to the canonical file system state after file metadata modification. This should hold regardless of the observed series of file changes and regardless of whether metadata for affected files was previously indexed.

- **Query efficiency:**

The index should be designed to allow a future query engine to efficiently find files based on their metadata. The Index structure should support efficient implementation of common comparison operators on individual attributes and conditional logic to negate or combine multiple comparison operations.

- **Resource efficiency:**

This daemon may run on battery-powered, resource constrained systems such as mobile devices, or on systems where foreground apps should have priority access to system resources. Therefore it must consume few system resources (CPU, memory and disk I/O footprint) when idle or waiting for file system events. Resource usage should likewise remain low when reacting to file system events, such as when collecting file metadata and updating index records so that the daemon's impact on foreground applications is minimised.

- **Timely index updates:**

Index updates should occur within a few seconds under typical system load and non-pathological update conditions, so the index feels responsive to users.

- **Minimal user maintenance:**

The daemon should operate autonomously: user intervention should not be required in normal daemon operation and it should gracefully recover from transient errors or inconsistent states wherever possible.

3.2 High-Level Architecture

The daemon is structured as a reactive pipeline which discovers new and/or updated files, collects their metadata and updates the index database accordingly. It comprises 4 main processing stages, spread across 2 threads:

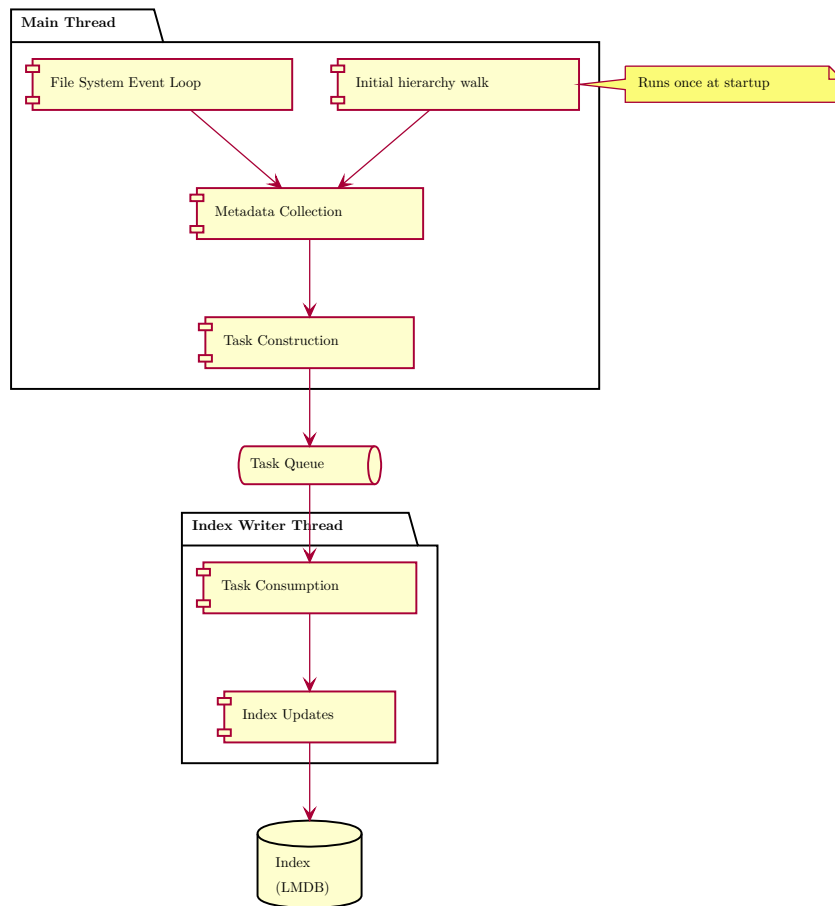


Figure 3.1: High-level architecture of the indexer daemon, showing the main and writer threads and their responsibilities.

On startup, the daemon performs the **initial hierarchy walk**. It recursively traverses the monitored hierarchy, discovering all files that must be indexed and gathering their metadata, in order to construct the index from an initially empty database.

After initial index construction is complete, the daemon sets up a fanotify instance monitoring the hierarchy, then enters the **file system event loop**, blocking waiting for file system events. When file creation or metadata change events are received, metadata for the affected files is rescanned and an index insert/update (upsert) task is queued. When file deletion events are received, delete tasks are queued to remove the corresponding metadata.

Per-file **metadata collection** is triggered by either of the above stages and consists of reading static file metadata such as file size, mtime and ctime timestamps, then reading values for all extended attributes the daemon is configured to index. From this data, an upsert task is constructed describing all attributes that need to be inserted or updated in the index.

Finally, **index updates** are performed based on the tasks constructed by metadata collection and file deletion, which contain all information necessary to update the index. All metadata associated with each file is inserted, updated or deleted in one atomic transaction, to maintain internal index consistency.

3.2.1 Concurrency Model

A dual-thread architecture is adopted to separate event ingestion and metadata collection from index maintenance. A global, bounded task queue is employed for inter-thread communication.

The **main thread** performs the initial scan, monitors file system events, collects metadata for created or modified files and enqueues upsert and delete tasks onto the queue. Its disk I/O workload consists mainly of reading file metadata; it performs very few write operations. It also handles graceful process termination, enqueueing a sentinel termination task when a termination signal is received.

The **index writer thread** dequeues tasks from the queue until it encounters the sentinel termination task, updating the index database accordingly. Its disk I/O workload consists solely of reading from and writing to the index database.

The threads communicate via a bounded task queue. The main thread produces tasks, which are consumed by the index writer thread. This decouples the rates of task production and consumption, which reduces the likelihood that one stage stalls the other and enables the daemon to absorb

bursts of file system activity. The queue is statically bounded to ensure that the indexer’s memory footprint doesn’t grow indefinitely. Bounding the queue size also provides backpressure: if the queue fills, the producer blocks; if empty, the consumer blocks awaiting new tasks.

The daemon must avoid overflowing the in-kernel queue of fanotify file system events, as this queue is bounded and overflow causes the kernel to drop events. An overflow would cause the real sequence of file system events to diverge from those observed by the indexer, leading to the index becoming inconsistent with the canonical file system state. Separating file system handling from index maintenance allows them to proceed simultaneously, which prevents index write operations from stalling the event loop under normal conditions and thus allows the indexer to read from the fanotify event queue more frequently.

This separation also isolates two distinct I/O patterns (metadata reads and database writes), which could allow per-thread I/O scheduling policies to be applied via use of the `ioprio_set` system call. [12] Doing so would enable the daemon’s I/O footprint to be controlled so that its impact on foreground applications can be managed. This remains a future research direction.

3.3 File System Scanning and Monitoring

Before event monitoring can commence, the daemon performs an initial recursive scan of the monitored hierarchy to build an index of baseline file system state. Starting at the monitored hierarchy’s root directory, a pre-order, depth-first traversal of all files is performed. [10] For each visited file, basic metadata (size, ctime and mtime) are requested, along with the values of all configured extended attributes (if present). The file’s absolute path and basename are also recorded. This metadata is finally packaged as an upsert task and enqueued for the index writer to process.

Once the hierarchy is scanned and indexed, the indexer begins watching for further file system mutations using the Linux kernel’s fanotify API. [8] Fanotify provides several granularities at which to monitor file system activity, including individual files and directories, mountpoints and entire file systems.

As discussed in section 2.2.2, directory monitoring is undesirable for metadata indexing as it only monitors the directory’s direct descendants; it does not recursively monitor files in subdirectories. While subdirectories could be recursively added to the notification group, this approach is susceptible to race conditions when directories are created, deleted and renamed during the scanning of each non-leaf directory. [7]

Mountpoint monitoring is likewise ineffective as file handle reporting and

required events, including file attribute changes, cannot be received with this mark type. [9]

Instead, the daemon monitors an entire file system, which both provides all event types required to reactively maintain the index and emits events for all files in the monitored hierarchy, including in subdirectories. The current prototype must therefore index all files on a file system; it does not support excluding files or directories, or indexing only a subdirectory hierarchy, though this could be implemented by using fanotify’s ignore mask to prevent excluded file system objects from generating events. [8] As file system monitors do not support passing file descriptors to identify files, file and directory ID reporting are enabled, causing file system events to carry opaque handles which identify files in a file system dependent manner (commonly by inode number). These are resolved with `open_by_handle_at`. [13] File handles may be stale by the time they are received (e.g. if the file was deleted); upsert tasks are not created for events with stale handles.

Paths must be resolved for each created file, to create an `id → path` mapping. Currently this is done by reading the symbolic link at `/proc/self/fd/N` where `N` is the file descriptor number, however this approach does not guarantee that the path points to the hard-link through which it was modified, or indeed that path is valid at all (e.g. it may contain “[deleted]” when no links to the file remain). Reconstructing the path by walking up the file’s parent hierarchy, using the parent directory handle from each fanotify event, would be a better approach, but this was not implemented due to time constraints.

For new or updated files, basic file metadata is collected with the `statx` system call; `xattr` values are retrieved with `lgetxattr`.

3.4 Storage Engine

The Lightning Memory-Mapped Database Manager, [’ 2] was chosen as the backing store, to minimise resource usage while maximising potential query performance.

Because LMDB uses ‘mmap’ to read and write the database, it “requires no page caching layer of its own,” [14] and takes advantage of kernel page caching instead. This gives the kernel transparency into how this memory is used, allowing it to reclaim cached database pages when that memory is required by another process such as a foreground program. Pages need only be flushed to the database file on disk if they are dirty.

Furthermore, LMDB provides ACID semantics: transactions allow us to atomically update all indexed metadata for a file at once, and studies show that the database remains consistent even after abrupt process termination.

[16, 19] Compounded by the fact that LMDB “requires no maintenance during operation” compared to other DBMSs such as SQLite that use write-ahead logging, [14] this makes it suitable for running on consumer devices where no system maintenance is expected.

Finally, because LMDB stores data in B-trees, if the data stored in the index is constructed to take advantage of B-tree ordering guarantees, many operators such as ‘==’, ‘<’, ‘>’, ‘<=’, ‘>=’, ‘starts with’, and ‘ends with’ can be implemented in $O(\log n)$ time complexity, making for fast index lookups.

LMDB is not without disadvantages: for example you must specify the maximum amount of address space that a mapped database can use, and LMDB cannot read or write outside this area. 64bit systems provide sufficient address space for an index of any reasonable size to be mapped, but 32bit systems only have 4GB of address space, some of which is kernel reserved. While 32bit CPUs are still common in embedded systems, they have largely disappeared from consumer devices like phones, tablets, laptops and desktops, the execution target for this indexing service. LMDB is additionally low-level: there is no query planner or query language; a future query engine must traverse the database using cursor operations.

Data is serialised in a machine-portable, binary fashion: unsigned integers are stored as fixed-width and in big-endian order so their order is preserved by mere byte-wise comparison; signed integers are not required and strings are stored and compared lexicographically as raw bytes.

3.5 Data Model

3.5.1 File Identity

While paths are the primary, user-facing way to identify files, they are not suitable as the primary key of a metadata index as using hard-links, multiple paths may reference the same underlying inode. Thus, the index references files via their inode number. These integer identifiers are unique at any given moment within a single file system, but may refer to different files in distinct file systems. As such, the current index design only permits indexing one file system per database. Cross file system support could be implemented by storing a compound primary key comprising the FS’s universally unique identifier (UUID) and the inode number.

3.5.2 Reverse Index Design

Reverse indexes facilitate efficient queries by supporting fast equality and range scans over file metadata values. All reverse metadata indexes comprise a compound key concatenating the metadata value with the file’s inode number. For example, the structure of a basename record is shown in table 3.1.

Table 3.1: Structure of a basename reverse index key

Offset (bytes)	Size (bytes)	Field
0	N	File basename string of length N, likely UTF-8
N	1	Nul-terminator (<code>\0</code>)
N+1	8	Inode number (unsigned 64bit integer)

Because LMDB sorts keys lexicographically, reverse indexes are therefore ordered by metadata values, making the average lookup time-complexity $O(\log n)$.

Reverse index values are unused and remain empty.

3.5.3 Forward Index Design

While Reverse indexes map attribute values to the files they belong to, forward indexes do the opposite, mapping inode numbers to all indexed metadata for that file. They are used to accelerate index entry update and delete operations by providing the information necessary to locate existing reverse index entries. Without forward index records, removing a reverse index entry would incur a linear ($O(n)$) search; with them the time-complexity is logarithmic.

The `id_meta` forward index maps inode numbers to an on-disk structure containing basic metadata. This structure is shown in listing 1.

The `id_xattr` forward index stores all indexed extended attribute key/value pairs for each file. A compound key is used concatenating an inode number to a nul-terminated xattr name; the xattr value is stored verbatim. This enables optimised lookup of a xattr value for a file and fast iteration of all xattrs for a file.

3.5.4 Index Maintenance

A write transaction is opened for each dequeued task. Consequently, file index updates are done atomically per received upsert or delete event, so it

```

// On-disk structure storing combined file metadata. This is used for reverse
// lookup (for instance when finding index records that belong to a renamed or
// deleted file so they can be updated or removed). All fields are stored as
// big-endian.
typedef struct {
    u32 magic; // FILE_STATE_MAGIC
    u32 mask; // Defines which fields are filled.//
               // struct.//
    u64 size; // FILE_STATE_HAS_SIZE mask bit
    u64 mtime; // FILE_STATE_HAS_MTIME mask bit.
    u64 ctime; // FILE_STATE_HAS_CTIME mask bit.
} FileMeta;

```

Listing 1: FileMeta, used as id_meta value structure

is impossible to observe a state in which the forward and reverse indexes show an inconsistent view of file metadata.

During an upsert task, the index writer thread retrieves currently indexed metadata for the affected file, comparing it to the new information gathered after the triggering file system event. Where information has been updated, forward indexes are used to locate the out-of-date reverse index records, then all relevant forward and reverse records are updated and the transaction is committed.

The reader's attention is drawn to the extended attribute update process: a diff algorithm is used to compare the old and new lists of xattrs in sorted order; the code iterates over both lists in tandem, detecting new, updated and removed attributes and updating the index accordingly.

To process a deletion task, the inode number and forward indexes are used to locate all metadata records for the affected file and these are deleted.

If an error occurs during an index update, a message is logged and the transaction is aborted. As all new metadata is fetched by the main thread and bounded arena allocations are used, the predominant class of errors that could cause transaction abortion are LMDB errors caused by underlying I/O subsystem failure.

Chapter 4

Evaluation

The evaluation of the prototype was designed to assess both the *correctness* of the indexed file metadata and the *resource usage* of the daemon during the initial metadata scan and subsequent file system event monitoring period. These criteria were chosen as the project prioritised reactive, timely index updates when file metadata changes; economical use of system resources even when updating the index in response to metadata changes; and aimed to be a light-weight service on relatively low power consumer equipment with a minimal impact on foreground application performance. Correctness checking of the index’s metadata relative to the oracle file system’s ground-truth metadata state was used to assess the prototype’s ability to create an index database during the initial scan, and to maintain said index as file metadata changed in an accurate, timely fashion. Resource usage measurement of the prototype over the process’s life-cycle was correlated with file system and disk I/O statistics and employed as a proxy metric to determine the operational cost to system resources of the indexing daemon and speculate on its impact on other running processes.

4.1 Evaluation Methodology

Evaluation was conducted by combining indexing of test datasets, controlled file system churn tests, monitoring of indexd process resource usage and post-run correctness verification of the generated metadata database. Varying workloads were executed against the file system being monitored by the daemon, designed to stress-test different paths of the prototype’s index update logic. Data was collected for later analysis by measuring daemon resource usage against these workloads and by determining the amount of indexed metadata consistent with the oracle file system data and the completeness of the index, both before and after large amounts of metadata

churn. This data was then analysed to quantify the footprint of the daemon and allowed the accuracy of data in its maintained index to be assessed.

4.2 Experimental Setup

All experiments were performed using the same operating system version on the same hardware, with datasets and update patterns dictated by seeded random generation. The supervisor interleaved runs using different datasets to break so that test results may be reproduced.

Evaluation was conducted on a Ubuntu 24.04 virtual machine running via KVM on a Proxmox VE host, with the following specs:

This was done as the VM provided a minimal environment with few background processes, minimising the probability of other programs interfering with the performance and resource usage data.

Datasets were generated deterministically by a purpose-built helper tool, and stored on an EXT4 file system in a mounted disk image. While this arrangement sacrifices performance by requiring two layers of I/O block indirection, the dataset file system was isolated in this way from the host’s file system to prevent unwanted indexing of other files, as the prototype can only monitor an entire file system. EXT4 was chosen as it is one of the most common Linux file systems, many consumer-facing distributions use it by default and it provides good extended attribute support.

These datasets were reused across test runs. The generator used the seed “3110037” to control the exact directory structure, file naming and extended attribute keys/values produced for each file. Generated files have no contents as the indexer only reads metadata.

Three datasets were generated of different sizes, to see how daemon footprint and index accuracy changed with scale, and are shown in table 4.1.

Table 4.1: Generated dataset configuration parameters

Dataset	Total Directories	Total Files	Max. Directory Depth	Misfiling Probability
small	28	100	3	0.06
medium	1600	10000	5	0.05
large	140000	1000000	7	0.04

A custom evaluation supervisor was created to orchestrate stress testing of the prototype; it was designed to manage the life-cycle of indexd and the workload and correctness checking processes, collect resource statistics and test results for later processing and provide a framework for maximising consistency and repeatability of the test results by ordering tests in such

a way as to minimise inter-run dependencies, such as kernel page caching behaviour. This supervisor provided a framework to test the prototype against a matrix of datasets and workloads in an automated fashion. The daemon was modified to emit life-cycle events such as `initial_walk_start`, `initial_walk_end`, `watching_start`, etc. For each run, the supervisor then consumed these timestamped events, spawning workloads to exercise the index once the `watching_start` event was received ¹.

Each long-running process the supervisor spawns, including `indexd` and the workloads ran against it, are monitored by a `pidstat` instance, which samples process resource usage metrics such as CPU usage, resident (memory) set size and voluntary/involuntary context switches; this data is also saved for later analysis.

Once all workloads were complete for this run, the supervisor spawned scripts to validate the correctness of the index. A manifest containing received `indexd` life-cycle events and the exit statuses of each workload and correctness check was then recorded.

Workloads were selected not to maximise file system metadata I/O throughput or latency, but to test the daemon's update paths for the specific metadata types the prototype indexes: `ctime`, `mtime`, file size and a configurable list of extended attributes. The aim was to evaluate the suitability of the indexer's algorithms used when files are created or deleted, or when file attributes change, on the grounds of resource overhead and accuracy of the database's forward and reverse indexes.

The following workloads were used:

- A **startup baseline**, where no load-tests were run, to establish initial scan performance and isolate idle watch costs;
- Workloads that **create** or **delete** many files, to exercise database insert and delete operations;
- A workload that affects the **ctime** attribute by modifying the permission bits of files (as changing *any* file metadata updates **ctime** on Linux);
- A workload using the **utimes** system call to churn the **mtime** metadata field;
- Workloads that affect a file's **size** by for example using the **fallocate** syscall;

¹Workloads are started at this point because the prototype does not perform the initial scan in parallel with its event monitoring, so the index may become out-of-date if the dataset is modified during the startup phase.

- A workload that churns **xattr values** for xattrs that indexd is configured to index.
- **Mixed workloads** combining multiple metadata churn workloads from above, to approximate more heterogeneous write patterns;
 - This was implemented via a custom script, as stress-ng’s xattr workload does not allow specifying a list of xattr names to create/update values for.

Most of these workloads were generated by stress-ng, as it is an existing stress testing framework that has many built-in, reproduceable file metadata stressers that are relevant to this project. Stress-ng was only used for workload generation; it’s own system resource measurement was foregone in favour of granular process-level statistics reporting, as managed by the evaluation supervisor.

Pidstat recorded the following information at a 1-second interval for the indexd process and additionally all workloads:

- **CPU utilisation:** percentage of CPU used by the application (in user mode) and the kernel on behalf of the application (in kernel mode);
- **Memory utilisation:** total memory consumed (virtual size) and number of major and minor page-faults (where major page faults require loading data from disk);
- **Disk (I/O) utilisation:** the rate at which the process is reading or writing data to disk (KB/s) and the average I/O delay between requesting and receiving data from disk;
- **Task switching activity:** The number of voluntary (caused by system call) and involuntary (caused by exhaustion of the process’s time-slice) context switches the process made per second.

Correctness was assessed by determining how close the final state of the index matched the state of the canonical metadata source (the file system) both after the initial scanning process and after workload[s] had generated many metadata change events. A checking script was written that verified the contents of the LMDB database against its respective dataset.

1. The script first walked `id_meta` (the forward index mapping file IDs to basic metadata), verifying that basic info such as `ctime`, `mtime` and `size` were accurate with respect to the file system.

2. It then walked `id_xattr` (the forward extended attribute index), once again verifying that all indexed xattr values matched the values stored by the file system.
3. The script then iterated over all reverse metadata indices, to ensure that no stale reverse index entries had been left behind by previous update operations.
4. Finally, the checking script walked the dataset’s directory hierarchy, querying the index for metadata of traversed files to verify that no files or file metadata was missing from the index.

Any inconsistencies found in the index were logged and collected by the supervisor for later analysis.

Evaluation runs were repeated 3 times by the supervisor to validate that data was consistent between runs. These repetitions were separated by a short cooldown period and interleaved with runs using other datasets to break any invisible dependencies between runs, such as those caused by data that is already in the kernel’s page cache.

The evaluation intensionally focused on a narrow scope of workloads, particularly those that provided good proxy statistics for quantifying the indexer’s performance against its stated objectives. These workloads were not exhaustive and may not always represent real-world file system access and modification patterns. Some workloads were intensionally left out due to known broken functionality in the prototype, such as file renaming, which was not fully implemented. Running such workloads would be superfluous as it is already known that they would cause index inconsistencies.

4.3 Correctness Results

A correctness checker program was written and can be found at `tools/check_correctness.py`. This checker was ran after every performance analysis test by the supervisor. The results indicate that the daemon largely keeps up with change events and maintains the index correctly, however, the ctime and mtime of parent directories is not updated when child files change, causing them to get out of sync.

4.4 Performance Results

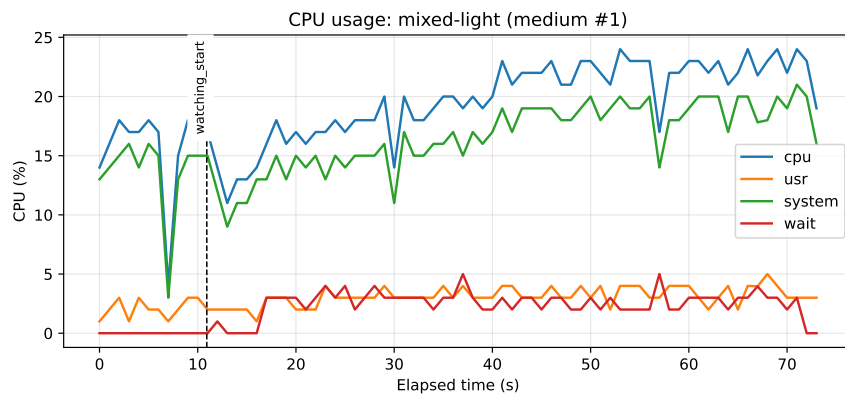
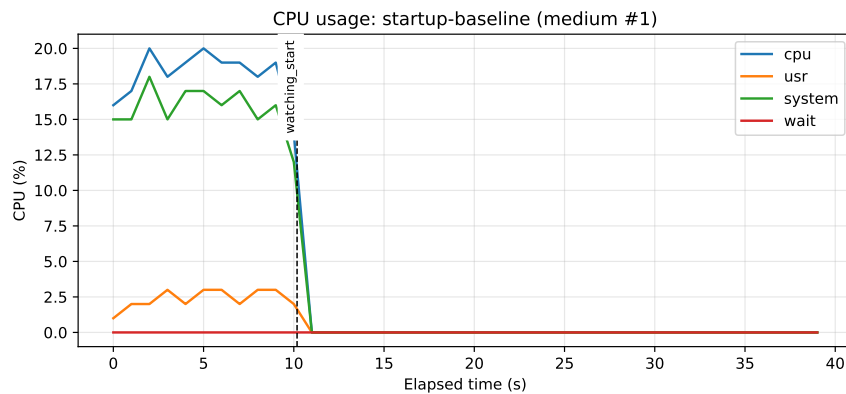
Empirical performance analysis data suggests that the indexing daemon indeed consumes few resources when blocked, waiting for events and that

as expected, resource usage increases under load. The data below compares resource usage of the daemon under a **startup baseline** (no load) vs a **mixed light** load comprising the following loads, run in parallel:

- create-grow
- delete-churn
- ctime-chmod
- mtime-utime
- size-fallocate
 - xattr-targeted

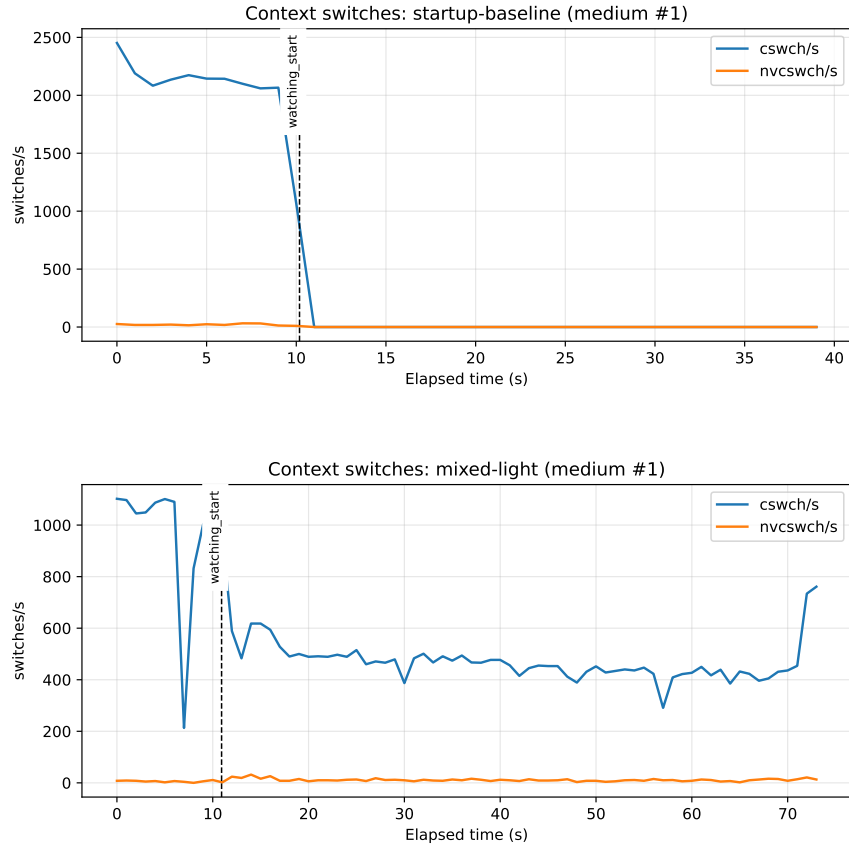
The complete dataset can be referenced in Appendix A.

4.4.1 CPU Usage



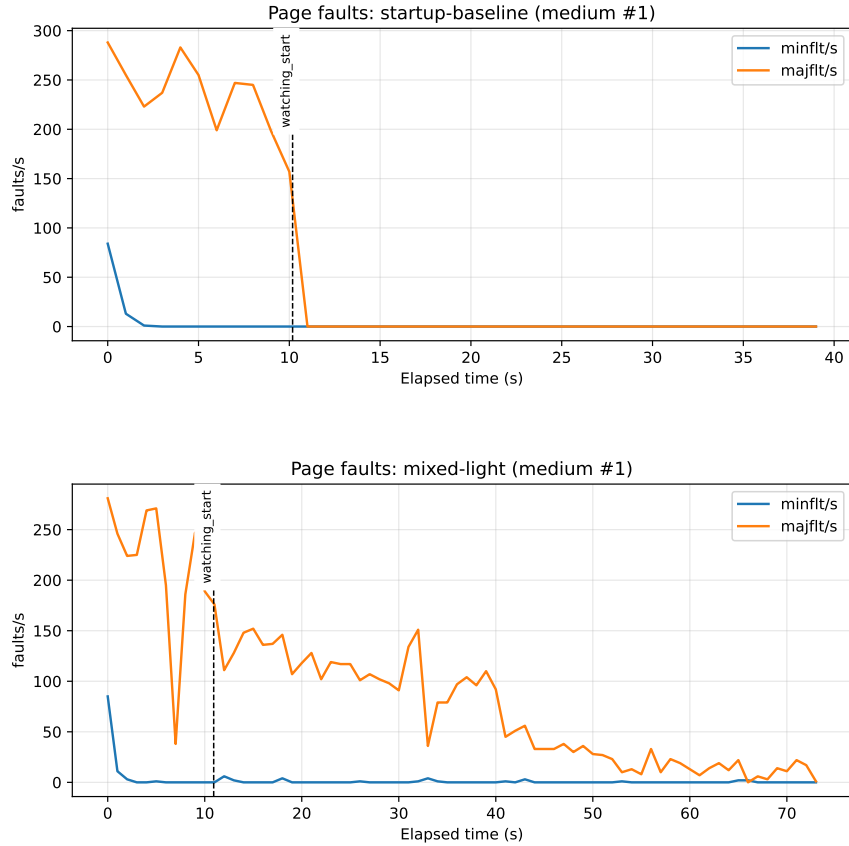
Against no load, CPU usage builds while constructing the index via the initial scan, dropping to lower levels once the watching phase starts. When the mixed load is ran, we can see that the daemon’s CPU usage remains higher while it handles constant index tupdates.

4.4.2 Context Switches



The data collected when the indexer was under no load confirms that the daemon achieves the goal of minimising wake-ups when in the watching phase, as no context switches happen once the initial index is constructed. By contrast, the many system calls made by collecting metadata and writing to the index explain why many more context switches are performed during the watching phase of the mixed load.

4.4.3 Memory Page Faults



The initial indexing phase shows here that when the indexd process first starts, hot database pages are not in the kernel page cache, however as the hierarchy walk continues, the cache begins to fill with commonly accessed pages. As the B-tree grows deeper, LMDB allocates more pages and thus page faults increase again. Meanwhile under a mixed load producing general file system activity, page faults remain at a reasonable level during the watching phase.

4.5 Critical Objective Evaluation

4.5.1 Change detection

The objective was to detect when files in the monitored hierarchy were created, deleted or renamed and when their metadata was modified. This was **largely achieved**: fanotify allows the daemon to receive file system

events, expeditiously after they occur and file creation, deletion and attribute changes for all files are handled. This was confirmed by end-to-end correctness testing (section 4.3) and during development using daemon logs. However, reacting to renamed files was not implemented due to time constraints.

4.5.2 Storage format

The objective was to design and implement a storage format for file metadata that was conducive to efficient queryability by future search engines and enabled reactive indexer updates.

While this has not been confirmed by empirical results, the chosen design should accelerate common queries that find files based on the value of one or more of their attributes, by employing per-attribute reverse indices keyed by value and stored in B+ trees, which offer logarithmic-time insertion, deletion and location of index entries and are well-suited to block-oriented storage. [15] Entries are lexicographically ordered so that common comparison operators are trivially implementable. Index maintenance is then optimised by using a forward index to enable $O(\log n)$ reverse index entry lookup when updating or deleting them.

Additionally, the use of LMDB supports efficient query execution even in the presence of ongoing concurrent index updates thanks to multi-version concurrency control; the index writer and concurrent readers cannot block each other. [2]

Overall, the database and schema **theoretically fully achieves** their design goals, though future empirical evaluation of query performance should be undertaken.

4.5.3 Manual scan

The objective was to implement an initial hierarchy scan to construct a baseline index upon which updates could be applied. This was **fully implemented** (see section 3.3) and correctness testing (section 4.3) confirms that it successfully builds a complete, accurate index for all files on the monitored file system.

4.5.4 Index reactivity

The objective was to maintain the index as files changed in a timely manner, based on received file system events. This has been **achieved**: performance analysis (see 4.4) shows the indexer is capable of keeping up with copious

synthetically generated file system events while keeping the index up-to-date.

4.5.5 Correctness

The objective stipulates that the index should be consistent with canonical file system state, regardless of the series of mutations performed. This has been **partially achieved**: section 4.3 shows that file creation, deletion and basic metadata modification events correctly update the index; the creation, modification and deletion of extended attributes is also successfully tracked. However, parental metadata updates (e.g. when a directory's mtime is updated as a result of an entry for a child file being created) are not handled, leading to inconsistencies in directory mtime and ctime index records.

4.5.6 Performance

Chapter 5

Conclusion

Recaps the work and key outcomes.

Reflects on the development process and lessons learned.

- Always thoroughly design your data model and database modification procedures before implementing them,
- File search is *extremely* hard to keep up to date in user-space because all event mechanisms are inherently lossy, and it is not hard to cause the indexer to lack the information it needs to successfully update its index.

5.1 Future Work

The indexer's current design and implementation has some limitations.

Rename events are not supported, meaning the index becomes inconsistent when files are renamed or moved. Fanotify provides `FAN_RENAME`, `FAN_MOVED_FROM` and `FAN_MOVED_TO` events; the daemon should react to these events by updating `id_path` and `basename` records for the target files. Supporting files with multiple hard links is more complex: files are referred to by inode number (see section 3.5.1) so indexed content is not duplicated, furthermore `id_path` allows multiple paths to be associated with a single inode, but when a delete task is processed, no paths are removed until the file's final link is unlinked, at which point all of them are removed.

Many useful file metadata fields remain to be indexed, such as file birth times, owner and group information or permission mode bits. Implementing these would be very similar to the existing `ctime`, `mtime` and `size` attributes.

Metadata change time could be exploited to reduce I/O operations during both the initial scan and monitoring phases, by only reindexing files whose ctime is later than the indexed ctime.

Richer queries may be enabled by supporting distinct, configurable reverse index types. For instance, storing reversed xattr values makes suffix queries efficient (perform a prefix lookup of the reversed suffix).

Perhaps the biggest limitation of the current design is fanotify itself. “Fanotify reports only events that a user-space program triggers through the filesystem API.” [8] Therefore it cannot detect modifications made by kernel subsystems such as NFS. By default, the size of the event queue is also limited and while this limit can be removed by a privileged process, doing so could facilitate a memory denial of service attack. Additionally, fanotify (indeed no Linux file event notification API) guarantees that target files of received events do not change between creation and consumption of the event; files can be changed, moved or even deleted between when their metadata is requested and when it is indexed. It is impossible to create an indexing service in user-space that guarantees consistency with the underlying file system state, thus, other indexing strategies should be explored, like constructing the index of metadata in the file system itself so that index updates may be part of the same transaction as the change to the file’s canonical metadata (via journalling or copy-on-write).

Bibliography

- [1] Neil Brown. “RCU-walk: faster pathname lookup in Linux”. In: *LWN.net* (July 2015). URL: <https://lwn.net/Articles/649729/> (visited on 04/09/2026).
- [2] Howard Chu. *MDB: A Memory-Mapped Database and Backend for OpenLDAP*. Paper. Symas Corp. and OpenLDAP Project, Oct. 2011. URL: <http://www.lmdb.tech/media/20120322-UKUUG-MDB-txt.pdf> (visited on 04/17/2026).
- [3] eBPF Community. *eBPF — Introduction, Tutorials & Community Resources*. 2026. URL: <https://ebpf.io/> (visited on 04/09/2026).
- [4] Free Software Foundation, Inc. *locate(1) – list files in databases that match a pattern*. GNU findutils project. Dec. 27, 2020. URL: <https://www.man7.org/linux/man-pages/man1/locate.1.html> (visited on 04/09/2026).
- [5] GNOME Project. *TinySPARQL: Overview*. 2024. URL: <https://tracker.gnome.org/overview/> (visited on 04/09/2026).
- [6] IEEE and The Open Group. *The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024. The Open Group Technical Standard Base Specifications, Issue 8*. The IEEE and The Open Group. 2024. URL: <https://pubs.opengroup.org/onlinepubs/9799919799/> (visited on 04/09/2026).
- [7] Michael Kerrisk. *Filesystem notification, part 2: A deeper investigation of inotify*. July 14, 2014. URL: <https://lwn.net/Articles/605128/> (visited on 04/14/2026).
- [8] Linux man-pages project. *fanotify – monitoring filesystem events*. Version 6.16. May 17, 2025. URL: <https://man7.org/linux/man-pages/man7/fanotify.7.html> (visited on 04/09/2026).
- [9] Linux man-pages project. *fanotify_mark – add, remove, or modify an fanotify mark on a filesystem object*. Version 6.16. May 17, 2025. URL: https://man7.org/linux/man-pages/man2/fanotify_mark.2.html (visited on 04/17/2026).

- [10] Linux man-pages project. *ftw, nftw – file tree walk*. Version 6.16. Sept. 21, 2025. URL: <https://man7.org/linux/man-pages/man3/nftw.3.html> (visited on 04/14/2026).
- [11] Linux man-pages project. *inotify(7) — monitoring filesystem events*. 2025. URL: <https://man7.org/linux/man-pages/man7/inotify.7.html> (visited on 04/09/2026).
- [12] Linux man-pages project. *ioprio_get, ioprio_set – get/set I/O scheduling class and priority*. Version 6.16. Sept. 21, 2025. URL: https://man7.org/linux/man-pages/man2/ioprio_set.2.html (visited on 04/11/2026).
- [13] Linux man-pages project. *name_to_handle_at, open_by_handle_at – obtain handle for a pathname and open file via a handle*. Version 6.16. Sept. 21, 2025. URL: https://www.man7.org/linux/man-pages/man2/open_by_handle_at.2.html (visited on 04/18/2026).
- [14] OpenLDAP Project. *LMDB — Lightning Memory-Mapped Database Manager Documentation*. URL: <http://www.lmdb.tech/doc/> (visited on 04/18/2026).
- [15] Andy Pavlo. *Database Storage I: Tree Indexes*. Lecture notes for 15-445/645: Database Systems. 2021. URL: <https://15445.courses.cs.cmu.edu/fall2021/notes/07-trees.pdf> (visited on 04/17/2026).
- [16] Thanumalayan Sankaranarayana Pillai et al. “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications”. In: *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. Broomfield, CO: USENIX Association, Oct. 2014, pp. 433–448. ISBN: 978-1-931971-16-4. URL: <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai>.
- [17] Stephen Rothwell. *Linux Directory Notification (dnotify)*. Version 6.19. Linux Kernel Documentation. URL: <https://www.kernel.org/doc/html/v6.19/filesystems/dnotify.html> (visited on 04/09/2026).
- [18] James A. Woods. “Finding Files Fast”. In: *login*: 8.1 (Jan. 1983), pp. 8–10, 15. URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/CSD-83-148.pdf> (visited on 04/09/2026).
- [19] Mai Zheng et al. “Torturing Databases for Fun and Profit”. In: *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. Broomfield, CO: USENIX Association, Oct. 2014, pp. 449–464. ISBN: 978-1-931971-16-4. URL: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_mai.
- [20] Yanliang Zou et al. “Stage Lookup: Accelerating Path Lookup using Directory Shortcuts”. In: *arXiv preprint arXiv:2010.08741* (Oct. 2020). DOI: [10.48550/arXiv.2010.08741](https://doi.org/10.48550/arXiv.2010.08741).